MUNICIPAL WASTE TRANSPORT PLANNING

NAGARFLOW AI

Project Master Guide

Architecture, local development, authenticated APIs, and delivery status for Scenarios, Depots, and Vehicles.

Prepared: October 31, 2026

Source of truth for setup and implemented behavior: repository README and code.

1. Product overview

NAGARFLOW AI is a React and FastAPI application for municipal waste transportation planning. Authenticated municipal coordinators manage planning scenarios and the configured depot and vehicle records used by those scenarios. The application is designed to use real database records; it does not seed demonstration depots, vehicles, facilities, or storage points.

2. Technology and repository layout

Frontend: React 18, React Router, and Vite under frontend/. Backend: FastAPI, Pydantic, SQLAlchemy 2, and Uvicorn under backend/. Database: PostgreSQL for local operation, with SQLite used by the backend test suite. API version prefix: /api/v1. Main application: backend/app/main.py. Frontend routes: frontend/src/App.jsx. Project setup and operational notes: README.md.

3. Local development setup

Requirements: Node.js and npm, Python 3.10 or later, and Docker Desktop when using the bundled PostgreSQL service. Copy .env.example to .env at the repository root and provide a secure JWT secret for any non-local deployment. Start the bundled database with: docker compose up -d db. The container maps PostgreSQL to host port 5433.

Backend PowerShell commands:
Set-Location backend
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt -r requirements-dev.txt
uvicorn app.main:app --reload

Frontend PowerShell commands in a second terminal:
Set-Location frontend
npm install
npm run dev

The API is served at http://localhost:8000 and interactive documentation is at http://localhost:8000/docs. The frontend defaults to the local API and accepts VITE_API_BASE_URL as an override.

4. Authentication and data access

The auth API supports registration, sign-in, current-user lookup, and password reset. Successful sign-in returns a short-lived bearer token. Protected scenario, depot, vehicle, and dashboard endpoints require authentication. Resource list and mutation endpoints scope data to the signed-in coordinator. Passwords are hashed; reset tokens are random, time-limited, stored as hashes, and single-use. In development, reset emails are written to backend/dev_emails/ when SMTP is not configured.

5. Step 5 - Scenario Management

The Scenarios page provides listing, creation, details, editing, and deletion. API: GET/POST /api/v1/scenarios, GET/PUT/DELETE /api/v1/scenarios/{id}, and GET /api/v1/scenarios/options. A scenario has a unique per-user name, description, active depot, processing facility, exactly two different available vehicles, six to ten different storage points, and Draft or Ready status. Selected resource IDs are persisted through scenario relationship tables. The API rejects unavailable vehicles, vehicles assigned to a different depot, and waste totals exceeding the selected vehicles' combined single-trip capacity. A scenario cannot be deleted while an optimization run is queued, pending, running, or in progress.

6. Step 6 - Depot Management

The Depot Management page provides listing, add, details, edit, and delete. API: GET/POST /api/v1/depots and GET/PUT/DELETE /api/v1/depots/{id}. Depot fields include name, unique code, address, latitude, longitude, and active status. Coordinates are range-checked. One active depot is allowed per coordinator. Vehicles require a configured active depot; inactive depots are excluded from new scenario choices. A depot linked to a scenario or assigned to a vehicle cannot be deleted. Existing database records are upgraded without manufacturing location data.

7. Step 7 - Vehicle Management

The Vehicles page provides listing, add, details, edit, and delete. API: GET/POST /api/v1/vehicles and GET/PUT/DELETE /api/v1/vehicles/{id}. Fields include unique registration number, display name, Truck or Mini Truck type, positive capacity, starting depot, and Available, Maintenance, or Inactive status. Capacity is stored as a positive decimal value. Vehicle starting coordinates follow the selected depot. Only available vehicles assigned to a scenario's selected depot can be selected. A scenario selects exactly two vehicles and cannot exceed their combined capacity. Vehicles linked to scenarios cannot be deleted or made unavailable until affected scenarios are updated. The fleet inventory may contain records beyond two; the two-vehicle rule is per scenario.

8. API quick reference

GET /health - service health.

POST /api/v1/auth/register - create account.

POST /api/v1/auth/login - sign in.

GET /api/v1/auth/me - current profile.

GET /api/v1/dashboard/summary - authenticated database-backed dashboard summary.

GET /api/v1/scenarios/options - real records available for scenario forms.

GET, POST /api/v1/scenarios - list or create scenarios.

GET, PUT, DELETE /api/v1/scenarios/{id} - read, update, delete scenarios.

GET, POST /api/v1/depots - list or create depots.

GET, PUT, DELETE /api/v1/depots/{id} - read, update, delete depots.

GET, POST /api/v1/vehicles - list or create vehicles.

GET, PUT, DELETE /api/v1/vehicles/{id} - read, update, delete vehicles.

All endpoints except health require bearer authentication, except the public auth flows.

9. Verification and tests

Backend tests use an in-memory SQLite database and do not require Docker or the development database. From backend/: py -m pytest. Frontend tests: from frontend/, npm test. Production build: from frontend/, npm run build. At the time this guide was generated, the full backend suite passed 86 tests, the frontend suite passed 33 tests, and the Vite production build succeeded. Tests cover authentication, CRUD, validation, relationship persistence, legacy schema upgrades, scenario selection, and fleet capacity constraints.

10. Data and operational safeguards

Never commit .env or real credentials. The repository ignores local environment files, node_modules, frontend build output, development email output, and database dumps. .env.example contains placeholders only. Do not populate resource tables with fake locations or capacities to make a form appear complete. Add genuine municipal records using the management pages. Keep JWT_SECRET_KEY unique and private outside local development.

11. Delivery roadmap

Completed modules: Step 5 Scenario Management, Step 6 Depot Management, and Step 7 Vehicle Management. Next planned module: Step 8 Storage Point Management. Later work includes processing facility management and route optimization. Scenario Management currently exposes only the route planning setup fields requested for Step 5; route optimization settings belong to a later module.